

B1229062-MCE-Lab5-Result

問題&心得:

- (1)

在這次實驗遇到的問題是一開始要讓實習板上的LCD亮起來還有印出字元時，一直不成功，後面改正插線後又只能亮基本LCD，然後用測試CODE還是印不出來，結果再思考嘗試是因為接線接反以及C語言撰寫錯誤造成的。
- (2)

還有在4*4按鍵上有因為接線失誤及配對port號對應數錯，後面修正後方可正常使用。
- (3)

bonus1無法向下滾動看先前字元紀錄核心原因:只能往上捲，不能往下捲，是因為我們的程式只有用line1和line2當兩行的暫存，每次捲動都會把上一行的內容直接覆蓋掉，舊資料沒有被記起來，所以沒有額外記憶體可以存"更早期的文字"，以及後續再做修改時一直出現空間不夠的error，以及明明編譯成功卻電腦當機找不到燒入的.hex檔，以上原因和時間不足而導致失敗。
- (4)

再來就是在bonus 2部分，我們原本的code會在按往上時或往下時無法做到正常滾動畫面，左邊到底也無法跳到上一行，右邊到底也無法跳到下一行，因為我們一開始的程式設計規劃是將往下滾動(按#)，和上(A)、下(B)、左(C)、右(D)鍵是分開的。

本次實驗內容結果:

1. lab05基本:

(1)

作法:

a.

先設LCD(8-bit、2-line)，並清除畫面，讓游標從第一列的開頭開始顯示字元。

b.

設定P2為Keypad(Column輸出、Row 輸入)，並用去彈跳的方式去偵測按鍵。

c.

主程式持續掃描Keypad，只要偵測到"剛按下"的鍵，就把對應自訂的ASCII顯示到LCD。

d.

利用計數器count來記錄已顯示字元數，當顯示到第20個字時自動換到第二列。

e.

超過 40 字就不再顯示(符合 2×20 LCD)，保持簡報要求的基本輸入顯示功能。
- (2)

程式撰寫重點:

a.

Keypad掃描採用column逐一拉低、row偵測方式，搭配delay5ms去做彈跳。

b.

用Keypad_wait_release()確保輸入為"短按一次"的操作，避免重複顯示。

c.

LCD指令分成Command()與Data()，透過RS/RW/EN控制資料和命令之模式。

d.

自動換行邏輯由count判斷:count==20，LCD移到第二列開頭。

e.

主迴圈不使用延遲，保持即時偵測按鍵並更新LCD顯示。

(3)

結果:

a.

Keypad按下的數字/字母/符號會正確顯示在LCD上。

b.

當輸入達20字後會自動換到第二列顯示。

c.

輸入滿40字後不再新增字元，LCD不會異常或跳行。

d.

透過去彈跳與"放開才算一次按鍵"，這樣顯示才會穩定、不會重複輸入。

●

code:

```
#include <reg51.h> // 載入8051的基本標頭檔

sbit RS = P1^0; // LCD的RS腳位 (控制資料或指令)
sbit RW = P1^1; // LCD的RW腳位 (讀或寫)
sbit EN = P1^2; // LCD的EN腳位 (觸發用)
sbit KC0 = P2^0; // Keypad column 0 (輸出)
sbit KC1 = P2^1; // Keypad column 1 (輸出)
sbit KC2 = P2^2; // Keypad column 2 (輸出)
sbit KC3 = P2^3; // Keypad column 3 (輸出)
sbit KR0 = P2^4; // Keypad row 0 (輸入)
sbit KR1 = P2^5; // Keypad row 1 (輸入)
sbit KR2 = P2^6; // Keypad row 2 (輸入)
sbit KR3 = P2^7; // Keypad row 3 (輸入)

void Delay1ms(){ // 做一個1ms的延遲
    unsigned int i;
    for(i=0;i<120;i++); // 簡單延遲用的空迴圈
}

void delay2ms(){ // 大概2ms延遲
    Delay1ms();
    Delay1ms();
}

void delay5ms(){ // 大概5ms延遲
    Delay1ms(); Delay1ms(); Delay1ms(); Delay1ms(); Delay1ms();
}

void delay40us(){ // 很短的延遲 (LCD需要)
    unsigned char i;
    for(i=0;i<20;i++); // 小的空迴圈
}

void Command(unsigned char cmd){ // 送LCD指令
    P0 = cmd; // 指令丟到P0
    RS = 0; // RS=0, 指令模式
    RW = 0; // RW=0, 寫入模式
    EN = 1; // EN拉高觸發
    delay40us(); // 等一下
    EN = 0; // EN收回
}

void Data(unsigned char d){ // 傳一個字元給 LCD 顯示
    P0 = d; // 把資料丟到P0
    RS = 1; // RS=1, 資料模式
    RW = 0; // RW=0, 寫入
    EN = 1; // EN=1做觸發
    delay40us(); // 等一下
    EN = 0; // EN=0 關閉
}

bit keypad_any_pressed(){ // 檢查有沒有人按著按鍵
    if(KR0==0 || KR1==0 || KR2==0 || KR3==0) // 有任何row被拉低
        return 1; // 有按
    return 0; // 沒按
}

void keypad_wait_release(){ // 等按鍵放開
    while(keypad_any_pressed()); // 等到完全沒人按
    delay5ms(); // 做一下去彈跳
}

unsigned char keypad_scan_once(){ // 掃描一次Keypad，回傳按鍵字元
    static unsigned char keypad[4][4] = { // key 對應表
        {'1','2','3','A'},
        {'4','5','6','B'},
        {'7','8','9','C'},
        {'*', '0', '#', 'D'}
    };
    unsigned char row, col;
    KR0 = KR1 = KR2 = KR3 = 1; // row先設為輸入 (拉高)
    for(col=0; col<4; col++){ // 逐一掃描 column
        KC0 = KC1 = KC2 = KC3 = 1; // 全部先拉高

        switch(col){ // 目前要測試哪一行
            case 0: KC0 = 0; break; // 測 column0
            case 1: KC1 = 0; break; // 測 column1
            case 2: KC2 = 0; break; // 測 column2
            case 3: KC3 = 0; break; // 測 column3
        }

        if(KR0==0){ row=0; goto pressed; } // 如果row=0被拉低，那就是按這格
        if(KR1==0){ row=1; goto pressed; }
        if(KR2==0){ row=2; goto pressed; }
        if(KR3==0){ row=3; goto pressed; }
    }

    return 0; // 沒按任何鍵
}

pressed:
    delay5ms(); // 去彈跳
    keypad_wait_release(); // 等放開
    return keypad[row][col]; // 回傳該格對應的ASCII字
}

void main(){ // 主程式開始
    unsigned char key; // 用來放掃描到的按鍵
    unsigned char count = 0; // 計算目前已經輸入的字 (最多40)

    delay5ms(); // LCD初始化前等一下
    Command(0x3F); // 設8-bit, 兩行模式
    Command(0x0E); // 開背光+游標
    Command(0x01); // 清除畫面
    delay2ms(); // 清除畫面要等一下
    Command(0x80); // 游標到第一列開頭

    while(1){ // 無窮迴圈，一直掃描Keypad
        key = keypad_scan_once(); // 掃一次按鍵

        if(key != 0){ // 如果真的有按鍵
            if(count < 40){ // 還沒超過40字
                Data(key); // 把字印到LCD
                count++; // 計數+1

                if(count == 20){ // 印到20字，自動換行
                    Command(0xC0); // 移到第二列開頭
                }
            }
        }
    }
}
```

2. bonus 1、2(以下內容是重點部分):

●

此次**bonus**只有成功做出**bonus 1**的游標可以向下移動去做向上滾動以及刪除字元的部分，以及**bonus2**可以在當前畫面做出是上下左右游標移動。但無法向下滾動去看之前的字元印出紀錄。

(1)

作法:

a.

新增移動游標的功能，在目前畫面中的兩行可以上下左右移動。

b.

支援插入式輸入: 游標停在哪，字就插到哪，不會覆蓋。

c.

多加三種功能鍵: *為退格(刪一個字元)、0為清除整個畫面的字元、#為捲動整頁(往上捲一行)。

d.

顯示時用line1/line2作為buffer，必要時會"重繪整個LCD"。

(2)

程式撰寫重點(不同於basic):

a.

新增四個游標的移動函式.Left/Righ /Up/Down。

b.

新增backspace(*)、clear all(0)、scroll(#)三項功能。

c.

每次修改內容後會呼叫LCD_redraw()來重畫整個畫面。

d.

Keypad的按鍵功能重新定義A/B/C/D為方向鍵(分別為左、右、上、下)。

(3) 結果:LCD可輸入多行文字, 超過兩行會自動往上捲動並保持內容正常顯示, 且可以在當前畫面中進行游標上下左右移動。

● code:

```
#include <reg51.h> // 引入8051的基本程式庫

sbit RS = P1^0; // LCD RS 腳, 決定是資料還是指令
sbit RW = P1^1; // LCD RW 腳, 決定是讀或寫
sbit EN = P1^2; // LCD EN 腳, 負責觸發 LCD 動作
sbit KC0 = P2^0; // Keypad column0 (輸出)
sbit KC1 = P2^1; // Keypad column1 (輸出)
sbit KC2 = P2^2; // Keypad column2 (輸出)
sbit KC3 = P2^3; // Keypad column3 (輸出)
sbit KR0 = P2^4; // Keypad row0 (輸入)
sbit KR1 = P2^5; // Keypad row1 (輸入)
sbit KR2 = P2^6; // Keypad row2 (輸入)
sbit KR3 = P2^7; // Keypad row3 (輸入)
unsigned char line1[20]; // LCD 第一行緩衝區 (20格)
unsigned char line2[20]; // LCD 第二行緩衝區 (20格)
unsigned char cursor_pos = 0; // 游標位置 (0~19第一行, 20~39第二行)

void Delay1ms(){ // 大概 1ms 延遲
    unsigned int i;
    for(i=0;i<120;i++);
}

void delay2ms(){ Delay1ms(); Delay1ms(); } // 2ms 延遲
void delay5ms(){ // 大概 5ms 延遲
    Delay1ms();Delay1ms();Delay1ms();Delay1ms();Delay1ms();
}

void delay40us(){ // 很短, LCD 需要的延遲
    unsigned char i;
    for(i=0;i<20;i++);
}

void Command(unsigned char cmd){ // LCD 指令輸出
    P0 = cmd; // 指令送到 P0
    RS = 0; // 指令模式
    RW = 0; // 寫模式
    EN = 1; // 拉高觸發
    delay40us(); // 等 LCD 吃指令
    EN = 0; // 拉低結束
}

void Data(unsigned char dat){ // LCD 顯示資料 (顯示字)
    P0 = dat; // 資料送到 P0
    RS = 1; // 資料模式
    RW = 0; // 寫入
    EN = 1; // 觸發
    delay40us(); // 小延遲
    EN = 0; // 收掉 EN
}

void LCD_redraw(){ // 把 line1/line2 整個重畫到 LCD
    unsigned char i;
    Command(0x80); // 第一行起點
    for(i=0;i<20;i++) Data(line1[i]); // 重畫第一行文字
    Command(0xC0); // 第二行起點
    for(i=0;i<20;i++) Data(line2[i]); // 重畫第二行文字
}

bit keypad_any_pressed(){ // 檢查是否有人按著按鍵
    if(KR0==0 || KR1==0 || KR2==0 || KR3==0)
        return 1; // 有按到
    return 0; // 全部都是1, 無按鍵
}

void keypad_wait_release(){ // 等使用者放開按鍵
    while(keypad_any_pressed()); // 等沒有按鍵為止
    delay5ms(); // 去彈跳
}

unsigned char keypad_scan_once(){ // 一次掃描 keypad
    static unsigned char keymap[4][4] = { // 按鍵對應表
        {'1','2','3','A'}, // A = left
        {'4','5','6','B'}, // B = right
        {'7','8','9','C'}, // C = up
        {'*','0','#','D'} // * = backspace, 0 = clear, # = scroll, D = down
    };
    unsigned char row, col;
    KR0 = KR1 = KR2 = KR3 = 1; // row設成輸入狀態
    for(col=0; col<4; col++){ // 逐欄掃描
        KC0=KC1=KC2=KC3 = 1; // 全部先拉高
        switch(col){ // 只拉低一欄
            case 0: KC0 = 0; break;
            case 1: KC1 = 0; break;
            case 2: KC2 = 0; break;
            case 3: KC3 = 0; break;
        }
        if(KR0==0){ row=0; goto pressed; } // 哪一列被拉低就是哪一列
        if(KR1==0){ row=1; goto pressed; }
        if(KR2==0){ row=2; goto pressed; }
        if(KR3==0){ row=3; goto pressed; }
    }
    return 0; // 沒按鍵
}

pressed:
    delay5ms(); // 去彈跳
    keypad_wait_release(); // 等放開
    return keymap[row][col]; // 回傳按鍵字符
}
```

```
void scroll(){ // 捲動 (把第二行往上推)
    unsigned char i;
    for(i=0;i<20;i++){
        line1[i] = line2[i]; // 第二行變第一行
    }

    for(i=0;i<20;i++){
        line2[i] = ' '; // 第二行清空
    }

    cursor_pos = 20; // 游標移到第二行開頭
    LCD_redraw(); // 整個 LCD 重畫
    Command(0xC0); // 移到第二行開頭
}

void handle_backspace(){ // 處理退格
    if(cursor_pos == 0) return; // 第一格不能再退
    cursor_pos--; // 游標往左退一格
    if(cursor_pos < 20)
        line1[cursor_pos] = ' '; // 第一行退格, 變成空白
    else
        line2[cursor_pos - 20] = ' '; // 第二行退格
    LCD_redraw(); // 重畫畫面
    if(cursor_pos < 20)
        Command(0x80 + cursor_pos); // 第一行游標位置
    else
        Command(0xC0 + (cursor_pos - 20)); // 第二行游標位置
}

void handle_clear_all(){ // 清除整個畫面
    unsigned char i;
    for(i=0;i<20;i++){
        line1[i] = ' '; // 第一行清空
        line2[i] = ' '; // 第二行清空
    }

    cursor_pos = 0; // 游標回左上角
    Command(0x01); // 清除 LCD
    delay2ms(); // 要等一下
    Command(0x80); // 游標移回第一行
}

void move_cursor_left(){ // 游標往左
    if(cursor_pos > 0) cursor_pos--; // 左移
    if(cursor_pos < 20)
        Command(0x80 + cursor_pos); // 第一行游標
    else
        Command(0xC0 + (cursor_pos - 20)); // 第二行游標
}

void move_cursor_right(){ // 游標往右
    if(cursor_pos < 39) cursor_pos++; // 右移
    if(cursor_pos < 20)
        Command(0x80 + cursor_pos); // 第一行游標
    else
        Command(0xC0 + (cursor_pos - 20)); // 第二行游標
}

void move_cursor_up(){ // 游標往上 (第二行 → 第一行)
    if(cursor_pos == 20) cursor_pos -= 20; // 減 20 回到第一行
    Command(0x80 + cursor_pos); // 更新游標位置
}

void move_cursor_down(){ // 游標往下 (第一行 → 第二行)
    if(cursor_pos < 20) cursor_pos += 20; // 加 20 到第二行
    Command(0xC0 + (cursor_pos - 20)); // LCD 游標
}

void main(){ // 主程式
    unsigned char key, i;

    for(i=0;i<20;i++){ // 初始化顯示緩衝區
        line1[i] = ' ';
        line2[i] = ' ';
    }

    cursor_pos = 0; // 游標從最前面開始
    delay5ms(); // LCD 初始化起手式
    Command(0x38); // 8-bit, 兩行
    Command(0x0E); // 開顯示+游標
    Command(0x06); // 自動游標右移
    Command(0x01); // 清除 LCD
    delay2ms(); // 游標到第一行開頭
    while(1) // 一直跑
    {
        key = keypad_scan_once(); // 讀一次按鍵
        if(key != 0) // 有按鍵才做事
        {
            if(key == ''){ handle_backspace(); continue; } // 退格鍵
            if(key == '0'){ handle_clear_all(); continue; } // 清除所有
            if(key == '#'){ scroll(); continue; } // 捲動
            if(key == 'A'){ move_cursor_left(); continue; } // 左
            if(key == 'B'){ move_cursor_right(); continue; } // 右
            if(key == 'C'){ move_cursor_up(); continue; } // 上
            if(key == 'D'){ move_cursor_down(); continue; } // 下
            if(cursor_pos == 20) Command(0xC0); // 自動換行
            else if(cursor_pos == 40) scroll(); // 滿了就捲動
            if(cursor_pos < 20){ // 第一行輸入字
                line1[cursor_pos] = key;
                Data(key);
                cursor_pos++;
            }
            else{ // 第二行輸入字
                line2[cursor_pos - 20] = key;
                Data(key);
                cursor_pos++;
            }
        }
    }
}
```

● 實驗照片:

